

Dev Machine Configuration

Application Generation Process — macOS Guide

Section 1: One-Time Software Install, GitHub Setup & Configuration

Homebrew

If you don't have Homebrew installed, get it first:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Step 1: Install Software

Git

Git comes pre-installed on macOS via Xcode Command Line Tools. To confirm or install, run:

```
xcode-select --install
```

Or install via Homebrew:

```
brew install git
```

Node.js

Download the macOS `.pkg` installer from <https://nodejs.org/en>. Or install via Homebrew:

```
brew install node
```

.NET 10 SDK

Go to <https://dotnet.microsoft.com/en-us/download/dotnet/10.0> and select the macOS tab. Choose the `.pkg` installer for your chip:

- Arm64 — M-series Macs (M1, M2, M3, M4)
- x64 — older Intel Macs

SQL Server — Mac Alternative (Docker)

Note: SQL Server LocalDB is Windows-only. Use Docker instead, which is the standard Mac dev approach.

First, install Docker Desktop from <https://docker.com>, then run:

```
# Intel Mac:
docker pull mcr.microsoft.com/mssql/server:2022-latest
docker run -e "ACCEPT_EULA=Y" -e "SA_PASSWORD=YourPassword123!" \
  -p 1433:1433 --name sqlserver -d \
  mcr.microsoft.com/mssql/server:2022-latest

# M-series Mac (ARM-compatible):
docker pull mcr.microsoft.com/azure-sql-edge
docker run -e "ACCEPT_EULA=1" -e "MSSQL_SA_PASSWORD=YourPassword123!" \
  -p 1433:1433 --name sqlserver -d \
  mcr.microsoft.com/azure-sql-edge
```

Note: Check with Alan Eicker to confirm the Docker connection string setup for the project template.

Homebrew (Recommended)

If you don't have Homebrew installed, get it first — it simplifies all of the above:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Step 2: Create a GitHub Personal Account

Until the team is on the MWS Enterprise environment, use your personal GitHub account.

- If you don't have an account, create one at <https://github.com>
- After creating your account, email **Alan Eicker** with the email address associated with your GitHub account. He will grant appropriate access for the solutions template in Section 2.

Step 3: Create a Personal Access Token in GitHub

1. Log in to <https://github.com/>
2. Navigate to <https://github.com/settings/tokens>
3. Click “Generate new token” and select “Generate new token (classic)”
4. Fill in the token settings:
 - a) Add a note: Claude development
 - b) Set expiration to 90 days
 - c) Under select scopes — check **Repo** and **Read:Packages**
5. Click “Generate token” at the bottom
6. Copy the token value immediately — **you won't be able to see it again**. Paste it into a text editor temporarily; you'll need it in the next steps.

Step 4: Create a .npmrc File

Open Terminal and run:

```
cd ~
touch .npmrc
```

Step 5: Edit the .npmrc File

Open it in TextEdit:

```
open -e ~/.npmrc
```

Paste the following two lines, replacing `[YOUR_GITHUB_TOKEN]` with the token from Step 3:

```
@alan-eicker:registry=https://npm.pkg.github.com
//npm.pkg.github.com/:_authToken=[YOUR_GITHUB_TOKEN]
```

Step 6: Configure Git

Run the following in Terminal:

```
git config --global user.name "[your GitHub username]"
git config --global user.email "[your GitHub email address]"
```

Section 2: One-Time Config for Every New Application

Step 1: Install the Solutions Template

Run these commands in Terminal before every new project to get the latest template:

```
npm cache clean --force
npm install -g @alan-eicker/northstar-cli
```

Note: No execution policy issues on Mac — that's a Windows-only concern. These commands should work directly.

Step 2: Create Your App Folder

Run the following in Terminal, replacing the placeholders:

```
mkdir -p ~/claude_apps
cd ~/claude_apps
nstar create [app_name]
```

When prompted:

- For **"Which type of GitHub account?"** — enter **1** (Personal)
- When prompted for your GitHub access token, paste the token from Section 1, Step 3. The token will not display — just paste and hit Enter.

After completion, your app folder will be created at `~/claude_apps/[app_name]`. Verify it exists using Finder.

Section 3: Claude Code — Requirements Gathering & Code Generation

Step 1: Set Working Folder in Claude Desktop

Launch Claude Desktop, go to the Code tab, and set the working folder to the app folder created in Section 2, Step 2.

Step 2: Run Requirements Gathering

```
/requirements-gathering
```

Follow the prompts and provide the requested information.

Step 3: Design Step (Currently Under Construction)

Skip this step for now and proceed to Step 4.

Step 4: Run /plan

```
/plan
```

This step plans the application architecture and implementation.

Step 5: Run /build

```
/build
```

This step builds the scaffold and generates code for the application.

Step 6: Run /ship

```
/ship
```

This step performs unit testing, code review, security audit, auto-remediation, and pushes the code to the GitHub repo.

Quick Reference: Mac vs Windows

Windows	macOS
PowerShell	Terminal
C:\Users\[user_name]	~ (home directory)
New-Item .npmrc	touch .npmrc
Set-ExecutionPolicy...	Not needed
SQL Server LocalDB installer	Docker + SQL Server container
.NET windows-x64 installer	.NET macOS .pkg (arm64 or x64)
File Explorer	Finder